

J.P. Morgan — SDE-1 (Java Backend) Masterclass

Complete Technical Guide: Algorithms, Core Java, Spring Boot, LLD & Architecture

Role: SDE-1 (Backend, Java) | Focus: Banking Tech, Concurrency & Distributed Systems

Prepared by: Shujat Khan

MASTER INDEX OF INTERVIEW QUESTIONS

Q#	Question Topic / Focus Area	Category
1	Find all Subarrays with Sum = K (Prefix Sum + HashMap)	DSA
2	Implement LRU Cache without using LinkedHashMap	DSA & LLD
3	Detect a Cycle in an Undirected Graph	DSA
4	Implement Kth Smallest Element in a BST	DSA
5	Minimum Window Substring Containing All Characters	DSA
6	HashSet vs LinkedHashSet vs TreeSet Internal Working	Core Java
7	CopyOnWriteArrayList: When to Use & When Not To	Core Java & Concurrency
8	ThreadLocal: Internal Mechanics & Pitfalls	Core Java & Concurrency
9	ForkJoinPool vs regular ThreadPoolExecutor	Core Java & Concurrency
10	Deadlock, Livelock, Starvation & Prevention	Core Java & Concurrency
11	CompletableFuture in Asynchronous Programming	Core Java & Concurrency
12	WeakReference vs SoftReference vs PhantomReference	Core Java & JVM
13	Spring Boot Auto-Configuration & DataSource Selection	Spring Boot & Microservices
14	Spring Boot Starter Parent & Dependency Management	Spring Boot & Microservices
15	Debugging Lazy Initialization Exceptions in JPA/Hibernate	Spring Boot & Databases
16	@Transactional Isolation Levels in Banking Systems	Spring Boot & Databases
17	Handling Distributed Transactions (SAGA Pattern & 2PC)	Spring Boot & Microservices
18	Role-Based Access Control (RBAC) in Spring Security	Spring Security
19	API Gateway Rate Limiting in Spring Cloud	Spring Cloud
20	Spring Cloud Circuit Breaker & Resilience4j Integration	Spring Boot & Microservices
21	Detecting & Fixing N+1 Query Problem in Spring Data JPA	Spring Boot & Databases
22	Configuring HikariCP Connection Pool for High-Throughput	Spring Boot & Databases
23	Graceful Shutdown in Spring Boot & Kubernetes Probes	Spring Boot & Kubernetes
24	Real-Time Stock Price Aggregator System Design	Low-Level Design
25	Distributed Cache Service Design (like Redis)	Low-Level Design
26	Chat Messaging System Design (1:1 + Group + Offline Delivery)	Low-Level Design

27	URL Shortener System Design (bit.ly scale)	Low-Level Design
28	Multi-Channel Notification System Design	Low-Level Design
29	Document Collaboration Tool Design (Google Docs Concurrency)	Low-Level Design

DETAILED SOLUTIONS & JAVA PROGRAMS

1. Find all Subarrays with Sum = K

Explanation: Uses prefix sum with a frequency HashMap to find subarrays equal to K in O(N) time.

Array: [1, 1, 1], K = 2 -> Prefix Sum tracking frequency map.

```
public class SubarraySumK {
    public static int subarraySum(int[] nums, int k) {
        Map<Integer, Integer> map = new HashMap<>();
        map.put(0, 1);
        int sum = 0, count = 0;
        for (int num : nums) {
            sum += num;
            if (map.containsKey(sum - k)) count += map.get(sum - k);
            map.put(sum, map.getOrDefault(sum, 0) + 1);
        }
        return count;
    }
}
```

2. Implement an LRU Cache without using LinkedHashMap

Explanation: Combines a HashMap with a custom Doubly Linked List to achieve O(1) get and put operations.

HashMap Key -> Doubly Linked List Node (Head = MRU, Tail = LRU)

```
public class LRUCacheCustom {
    class Node { int key, val; Node prev, next; Node(int k, int v){key=k; val=v;} }
    private final int cap; private final Map<Integer, Node> map = new HashMap<>();
    private final Node head = new Node(0,0), tail = new Node(0,0);
    public LRUCacheCustom(int capacity) { cap = capacity; head.next = tail; tail.prev = head; }
    public int get(int key) {
        if(!map.containsKey(key)) return -1;
        Node node = map.get(key); remove(node); insertToHead(node); return node.val;
    }
    public void put(int key, int value) {
        if(map.containsKey(key)) remove(map.get(key));
        if(map.size() == cap) { map.remove(tail.prev.key); remove(tail.prev); }
        Node node = new Node(key, value); insertToHead(node); map.put(key, node);
    }
    private void remove(Node node) { node.prev.next = node.next; node.next.prev = node.prev; }
    private void insertToHead(Node node) { node.next = head.next; node.prev = head; head.next.prev = node; head.next =
    }
}
```

3. Detect a Cycle in an Undirected Graph

Explanation: DFS traversal keeping track of the parent node to detect back-edges indicating cycles.

Node 0 -> Node 1 -> Node 2 -> Node 0 (Back edge detected -> Cycle present)

```
public class GraphCycleDetection {
    public boolean isCyclic(int v, List<List<Integer>> adj) {
        boolean[] vis = new boolean[v];
        for (int i = 0; i < v; i++) {
            if (!vis[i]) {
                if (dfs(i, -1, vis, adj)) return true;
            }
        }
        return false;
    }
    private boolean dfs(int curr, int parent, boolean[] vis, List<List<Integer>> adj) {
        vis[curr] = true;
        for (int neighbor : adj.get(curr)) {
            if (!vis[neighbor]) {
                if (dfs(neighbor, curr, vis, adj)) return true;
            } else if (neighbor != parent) return true;
        }
        return false;
    }
}
```

4. Kth Smallest Element in a BST

Explanation: Inorder traversal of a BST visits nodes in ascending sorted order. Stop at the Kth visited node.

BST Inorder: [1, 2, 3, 4] -> K=2 gives 2.

```
public class KthSmallestBST {
    private int count = 0, result = -1;
    public int kthSmallest(TreeNode root, int k) {
        inorder(root, k);
        return result;
    }
    private void inorder(TreeNode node, int k) {
        if (node == null) return;
        inorder(node.left, k);
        count++;
        if (count == k) { result = node.val; return; }
        inorder(node.right, k);
    }
}
```

5. Minimum Window Substring

Explanation: Sliding window approach tracking character frequency requirements to find the smallest substring containing all characters of pattern T.

```
S = "ADOBECODEBANC", T = "ABC" -> Window "BANC"
```

```
public class MinWindowSubstring {
    public String minWindow(String s, String t) {
        if (s.length() == 0 || t.length() == 0) return "";
        Map<Character, Integer> dictT = new HashMap<>();
        for (char c : t.toCharArray()) dictT.put(c, dictT.getOrDefault(c, 0) + 1);
        int required = dictT.size(), formed = 0, l = 0, r = 0;
        int[] ans = {-1, 0, 0};
        Map<Character, Integer> windowCounts = new HashMap<>();
        while (r < s.length()) {
            char c = s.charAt(r);
            windowCounts.put(c, windowCounts.getOrDefault(c, 0) + 1);
            if (dictT.containsKey(c) && windowCounts.get(c).intValue() == dictT.get(c).intValue()) formed++;
            while (l <= r && formed == required) {
                if (ans[0] == -1 || (r - l + 1) < ans[0]) { ans[0] = r - l + 1; ans[1] = l; ans[2] = r; }
                char ch = s.charAt(l);
                windowCounts.put(ch, windowCounts.get(ch) - 1);
                if (dictT.containsKey(ch) && windowCounts.get(ch).intValue() < dictT.get(ch).intValue()) formed--;
                l++;
            }
            r++;
        }
        return ans[0] == -1 ? "" : s.substring(ans[1], ans[2] + 1);
    }
}
```

6. HashSet vs LinkedHashSet vs TreeSet

Explanation: `HashSet` uses a HashMap with no ordering guarantee ($O(1)$). `LinkedHashSet` uses a doubly linked list maintaining insertion order ($O(1)$). `TreeSet` uses a Red-Black tree maintaining sorted order ($O(\log N)$).

```
// Set Implementations: HashSet (Unordered) vs LinkedHashSet (Insertion Order) vs TreeSet (Sorted)
```

7. CopyOnWriteArrayList: When to Use & When Not To

Explanation: Creates a fresh copy of the underlying array upon every mutative operation (`add`, `set`). Ideal for read-heavy, low-write concurrency. Terrible for frequent writes due to $O(N)$ copy overhead.

```
// Concurrency: CopyOnWriteArrayList for thread-safe iterator traversal without ConcurrentModificationException
```

8. ThreadLocal: Internal Mechanics & Pitfalls

Explanation: Binds thread-specific variables using a `ThreadLocalMap` stored inside each `Thread`. Pitfall: Memory leaks in thread pools if `remove()` is not called explicitly.

```
Thread -> ThreadLocals (ThreadLocalMap) -> Entry (WeakReference, Object Value)
```

```
public class TenantContext {
    private static final ThreadLocal<String> tenantId = new ThreadLocal<>();
    public static void set(String id) { tenantId.set(id); }
    public static String get() { return tenantId.get(); }
    public static void clear() { tenantId.remove(); }
}
```

9. ForkJoinPool vs ThreadPoolExecutor

Explanation: `ForkJoinPool` is designed for work-stealing algorithms where worker threads split large tasks (`RecursiveTask`) and steal subtasks from busy threads' queues. Standard `ThreadPoolExecutor` uses a shared blocking queue.

```
// Concurrency: ForkJoinPool optimized for divide-and-conquer parallel computing tasks
```

10. Deadlock, Livelock, Starvation & Prevention

Explanation: Deadlock occurs when threads wait indefinitely for circular locks. Prevented by establishing strict global lock ordering and timeouts.

```
Thread 1 (Lock A -> waits for B) <== DEADLOCK ==> Thread 2 (Lock B -> waits for A)
```

```
// Prevention: Consistent lock acquisition order across all multi-threaded service components
```

11. CompletableFuture in Asynchronous Programming

Explanation: Enables non-blocking asynchronous pipeline chaining (`supplyAsync`, `thenApply`, `thenCombine`, `allOf`) without blocking worker threads.

```
CompletableFuture.supplyAsync(() -> fetchAccount())
    .thenCombine(CompletableFuture.supplyAsync(() -> fetchLedger()), (acc, led) -> acc + led);
```

12. WeakReference vs SoftReference vs PhantomReference

Explanation: `SoftReference` objects are collected only when JVM runs critically low on memory (good for caches). `WeakReference` objects are collected at the next GC cycle (used in `WeakHashMap`). `PhantomReference` tracks object finalization cleanup state.

```
// Reference Types: Soft (Memory-sensitive cache) vs Weak (Canonical mappings) vs Phantom (Cleanup tracking)
```

13. Spring Boot Auto-Configuration & DataSource Selection

Explanation: `@EnableAutoConfiguration` scans classpath, inspecting libraries like HikariCP or Tomcat JDBC to instantiate the appropriate `DataSource` bean automatically.

```
// Auto-Configuration: Conditional bean instantiation detecting HikariCP on classpath
```

14. Spring Boot Starter Parent & Dependency Management

Explanation: `spring-boot-starter-parent` provides pre-configured Maven defaults, compiler versions, plugin management, and curated dependency version alignments.

```
// Build Management: Centralized dependency versioning via spring-boot-dependencies bom
```

15. Debugging Lazy Initialization Exceptions in JPA/Hibernate

Explanation: Occurs when accessing lazy-loaded entity associations outside an active Hibernate session. Resolved using `@EntityGraph`, `JOIN FETCH` queries, or Open Session in View.

```
// JPA Query: @Query("SELECT u FROM User u JOIN FETCH u.orders WHERE u.id = :id")
```

16. @Transactional Isolation Levels in Banking Systems

Explanation: `READ_COMMITTED` prevents dirty reads. `REPEATABLE_READ` prevents non-repeatable reads. `SERIALIZABLE` prevents phantom reads by locking data ranges. Essential for financial ledgers.

```
@Transactional(isolation = Isolation.REPEATABLE_READ)
public void transferFunds(Long from, Long to, BigDecimal amount) { ... }
```

17. Handling Distributed Transactions (SAGA Pattern & 2PC)

Explanation: 2PC (Two-Phase Commit) locks resources across distributed nodes (hurting availability). SAGA executes local transactions sequentially via choreography or orchestration with compensating rollbacks.

```
Step 1: Debit Wallet -> Step 2: Reserve Ticket -> [If Fail] -> Compensating Wallet Refund
```

```
// SAGA Pattern: Event-driven local transactions with automated compensating rollbacks
```

18. Role-Based Access Control (RBAC) in Spring Security

Explanation: Configured via Spring Security method security (`@PreAuthorize("hasRole('ADMIN')")`) or HTTP security matcher rules.

```
http.authorizeHttpRequests(auth -> auth.requestMatchers("/api/admin/**").hasRole("ADMIN"));
```

19. API Gateway Rate Limiting in Spring Cloud

Explanation: Configured at the Spring Cloud Gateway level using Redis rate limiters (`RequestRateLimiter` gateway filter) executing token bucket algorithms.

```
Client -> Spring Cloud Gateway (Redis RateLimiter Filter) -> Microservices
```

```
// Gateway Filter: Redis-backed reactive token bucket rate limiter
```

20. Spring Cloud Circuit Breaker & Resilience4j

Explanation: Wraps downstream service calls with circuit breakers to prevent cascading failures during partial outages.

```
API Call -> Resilience4j Circuit Breaker -> Downstream Microservice
```

```
// Resilience4j: Fallback execution when downstream microservices timeout or throw exceptions
```

21. Detecting & Fixing N+1 Query Problem in Spring Data JPA

Explanation: Occurs when fetching 1 parent entity triggers N additional queries for child collections. Fixed using `JOIN FETCH` or `@EntityGraph`.

```
// Fix: Use JOIN FETCH in JPQL queries to retrieve parent and children in a single SQL round-trip
```

22. Configuring HikariCP Connection Pool

Explanation: Tuned via `maximumPoolSize = (Core\_Count \* 2) + Effective\_Spindle\_Count`, ensuring connection pool exhaustion is prevented under high concurrency.

```
# HikariCP Tuning: spring.datasource.hikari.maximum-pool-size=20
```

23. Graceful Shutdown in Spring Boot & Kubernetes Probes

Explanation: Enabled via `server.shutdown=graceful`. Kubernetes readiness probes route traffic away from terminating pods while active incoming requests finish executing.

```
K8s Readiness Probe (Down) -> Stop Accepting Traffic -> Finish Active Requests -> Pod Terminate
```

```
# application.yml: server.shutdown=graceful & spring.lifecycle.timeout-per-shutdown-phase=30s
```


24. Real-Time Stock Price Aggregator System Design

Scenario: Ingesting millions of stock ticker price updates per second and calculating real-time VWAP.

Explanation: Kafka ingestion stream -> Apache Flink stateful stream processing engine -> Redis caching layer -> WebSockets for real-time client ticker broadcasts.

```
Stock Ticker Feeds -> Kafka -> Apache Flink (VWAP Windows) -> Redis Cache -> WebSockets
```

```
// Real-Time System: Low-latency stream processing pipeline handling millions of updates/sec
```

25. Distributed Cache Service Design (like Redis)

Scenario: Building an in-memory key-value data store supporting replication and high throughput.

Explanation: Single-threaded event loop executing non-blocking I/O multiplexing ('epoll'), consistent hashing across cluster nodes, and asynchronous master-replica replication.

```
Client Requests -> Epoll Event Loop (Single-Threaded Core) -> In-Memory Hash Table -> Replication Stream
```

```
// In-Memory Design: Non-threaded event loop architecture with O(1) hash table indexing
```

26. Chat Messaging System Design (1:1 + Group + Offline Delivery)

Scenario: Supporting persistent messaging with delivery acknowledgements and offline sync.

Explanation: WebSocket connections for active chat sessions, Kafka message queues for delivery fan-out, Cassandra/DynamoDB for high-throughput message history storage.

```
Client WebSocket -> Gateway -> Kafka Message Broker -> Cassandra (History) & Push/Offline Store
```

```
// Chat Architecture: Persistent WebSocket connections backed by Cassandra write-optimized storage
```

27. URL Shortener System Design (bit.ly scale)

Scenario: Generating compact short links and redirecting millions of clicks per second.

Explanation: Snowflake ID generator / distributed counters encoded in Base62 ('[a-z, A-Z, 0-9]'), backed by Redis caching for instantaneous redirects.

```
Long URL -> Hash / Snowflake ID -> Base62 Encoding -> Short URL -> Redis Redirect Cache
```

```
// URL Shortener: Base62 encoding mapping unique 64-bit integer IDs to 7-character strings
```

28. Multi-Channel Notification System Design

Scenario: Sending alerts via Email, SMS, and Push simultaneously with deduplication and retries.

Explanation: Event-driven architecture where notification requests publish to Kafka, fan out to channel-specific worker pools, and integrate with external vendor SDKs (SendGrid, Twilio, FCM).

```
Notification Request -> Kafka Topic -> Channel Routers (Email/SMS/Push Workers) -> Vendor APIs
```

```
// Notification Architecture: Decoupled pub/sub worker pools routing alerts across multi-vendor gateways
```

29. Document Collaboration Tool Design (Google Docs Concurrency)

Scenario: Real-time concurrent multi-user document editing without formatting collisions.

Explanation: Operational Transformation (OT) or Conflict-Free Replicated Data Types (CRDTs) managing concurrent character insertions and deletions across web clients via WebSockets.

```
Client A & Client B -> WebSocket Server -> CRDT / Operational Transformation Engine -> Document State Sync
```

```
// Collaborative Editing: CRDT / Operational Transformation resolving concurrent document state modifications
```